Pragya Singhal ,CU24220050

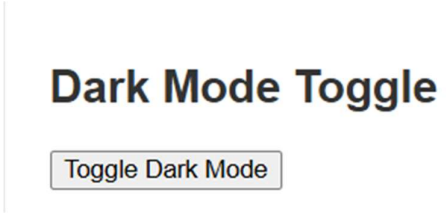
Assignment 2

1.**HTML**

```
<nav class="navbar">
  <div class="logo">MyBrand</div>
  <ul class="nav-links">
    <li><a href="#">Home</a></li>
    <li><a href="#">About</a></li>
    <li><a href="#">Contact</a></li>
  </ul>
</nav>
```

CSS

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 2rem;
  background-color: #333;
  color: #fff;  }
.nav-links {
  display: flex;
  gap: 2rem;
  list-style: none; }
.nav-links a {
  color: #fff;
  text-decoration: none;
}
```



2. const button = document.querySelector('.toggle-btn');

button.addEventListener('click', () => {

document.body.classList.toggle('dark-mode');

});

The method name addEventListner contains a typo (missing the e before ner). The correct standard DOM method name is addEventListener.

3. Vanilla JavaScript Signup Form Validation

Details / How it works: Upon the form submission event, `event.preventDefault()` is used to first validate the email and password. If the email lacks an '@' symbol or the password is shorter than 8 characters, an error message is displayed and the submission is halted.

<form id="signupForm">

<input type="email" id="email" placeholder="Email" required />

<input type="password" id="password" placeholder="Password (min 8 chars)" required />

<button type="submit">Sign Up</button>

</form>

4. Explanation: Repeated execution of the function causes new anonymous event listeners to be added to memory without the old ones being removed, resulting in a memory leak.

function handleScroll() {

console.log(window.scrollY);

}

// Purana listener remove karke naya lagane se memory leak nahi hoti

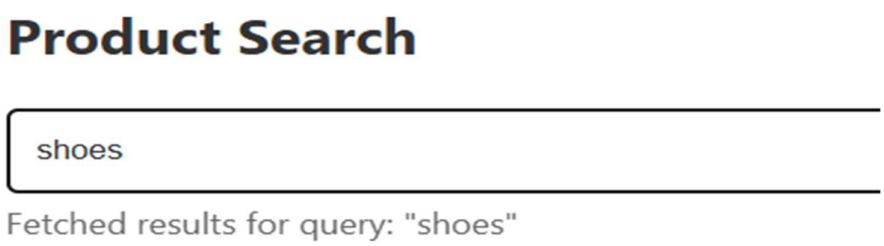
window.removeEventListener('scroll', handleScroll);

window.addEventListener('scroll', handleScroll);



5. Detail / How it works: Instead of making an API call immediately on every keystroke, the debouncing function sets a timer (`setTimeout`). The `searchProducts(query)` function is called only 400ms after the user stops typing.

```
function debounce(func, delay) {
  let timeoutId;
  return function (...args) {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => {
      func.apply(this, args);
    }, delay);
  };
function searchProducts(query) {
  console.log("API Searching for:", query);
}
const handleSearch = debounce((event) => {
  searchProducts(event.target.value);
}, 400);
// Input field par apply karna
// <input type="text" id="searchInput" placeholder="Search..." />
const searchInput = document.getElementById('searchInput');
if (searchInput) {
  searchInput.addEventListener('input', handleSearch);
}
```



6. .container {

display: flex;

width: 960px; }

.card {

width: 300px; }

/* Mobile Fix */

@media (max-width: 480px) {

.container {

width: 100%;

flex-direction: column; }

.card {

width: 100%; } }

7. Detail / Implementation: When the theme is toggled, either 'dark' or 'light' is saved using `localStorage.setItem('theme')`, and upon page reload, the theme is restored via the `DOMContentLoaded` event using `localStorage.getItem('theme')`.

// Theme save karna

function saveTheme(theme) {

localStorage.setItem('theme', theme);

document.body.className = theme;

}

// Page load hone par theme restore karna

window.addEventListener('DOMContentLoaded', () => {

const savedTheme = localStorage.getItem('theme') || 'light';

document.body.className = savedTheme;

});

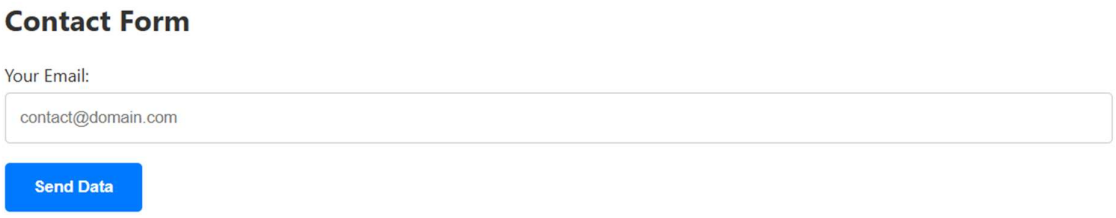
8. Details / How to do it: To prevent the page from refreshing upon form submission, use `e.preventDefault()`, then send the data to the server asynchronously using `fetch()`.

HTML

```
<form id="contactForm">
  <input name="email" type="email" required />
  <button type="submit">Send</button>
</form>
```

Javascript

```
const contactForm = document.getElementById('contactForm');
contactForm.addEventListener('submit', async (e) => {
  e.preventDefault(); // Page refresh hone se rokta hai
  const formData = new FormData(contactForm);
  const data = Object.fromEntries(formData.entries());
  try {
    const response = await fetch('/api/submit', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(data)
    });
    const result = await response.json();
    console.log('Success:', result);
  } catch (error) {
    console.error('Error:', error);
  }   });
```



9. Identification: Both `fetch()` and `.json()` are asynchronous operations that return a Promise; therefore, the `await` keyword was missing before them.

```
async function getUser(id) {
  const res = await fetch(`/api/users/${id}`); // await lagaya
  const data = await res.json();            // await lagaya
  return data; }
```

10. A gallery can be made responsive without any media queries by using `grid-template-columns: repeat(auto-fit, minmax(200px, 1fr))`. It automatically adjusts to 4 columns on desktop and 2 or 1 column on mobile.

HTML

```
<div class="gallery">
  <div class="card">Image 1</div>
  <div class="card">Image 2</div>
  <div class="card">Image 3</div>
  <div class="card">Image 4</div>
</div>
```

CSS

```
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 1rem; }
.card {
  background: #007bff;
  color: white;
  padding: 2rem;
  text-align: center;
  border-radius: 4px;  }
```

